Small-Scale Proxies for Large-Scale Transformer Training Instabilities

MVA 2025–2026

Tristan Beruard & Fotios Kapotos — CentraleSupélec / MVA — *Based on Wortsman et al., 2023 (Google DeepMind)*

Why Does This Matter?

Training very large Transformers is expensive — a single run can cost millions of dollars and weeks of compute. When a model with **tens of billions of parameters** diverges midway through training, the research team has lost an enormous amount of resources with little diagnostic information.

The Core Problem

Large runs fail in ways that *cannot be reproduced* at smaller scale with standard hyperparameters. Root-causes are therefore very hard to isolate and fix before the next expensive run.

The Key Insight

The Proposal

High learning rate acts as a proxy for **large scale**: the same instabilities that emerge at billions of parameters can be reliably reproduced in models with just a few million parameters, simply by pushing the learning rate into an extreme regime.

This makes instabilities *cheap to study, ablate, and fix.*

Paper Overview

Step 1 : Identify instabilities

Two known failure modes first seen in billion-parameter runs:

- Attention logit growth
- Output logit divergence

Both reproduced in small models at high learning rates.

Step 2 : Measure sensitivity

Introduce **LR sensitivity** $\mathcal{S}$:

$$\mathcal{S} = \mathbb{E}_{\eta\in[a,b]}\big[\min(\ell(A(\eta)), \ell_0) - \ell^*\big]$$

How much does the final loss degrade when you pick the wrong learning rate? A high $\mathcal{S}$ = fragile model.

Step 3 : Verify mitigations

Test the proposed fixes:

- **qk-layernorm**: LayerNorm on queries & keys before dot-product
- **z-loss**: auxiliary penalty $\alpha \log^2 Z$ on partition function

Both reduce $\mathcal{S}$ at small *and* large scale.

Our Methodology

Experimental setup

- Decoder-only Transformer based on **nanoGPT**, configured with Hydra
- Compact regime: **small models** used as proxies for large-scale failures
- Architecture close to the paper: rotary embeddings, no dropout, bias-free linear layers, no weight tying
- Objective: standard **next-token prediction**

Training protocol

- OpenWebText-style dataset from our pipeline
- Batch size **32**, sequence length **256**, one gradient accumulation step
- Optimiser: **AdamW** with $\beta_1 = 0.9$, $\beta_2 = 0.95$, bfloat16 when available
- Stress test via **learning-rate sweeps** and ablations of qk-layernorm, z-loss, and weight decay

Instability #1 — Attention Logit Growth

Mechanism

In self-attention, the logit between position i and j in a head of dimension d_h is:

$$z_{ij} = \frac{\langle q_i,\, k_j \rangle}{\sqrt{d_h}}$$

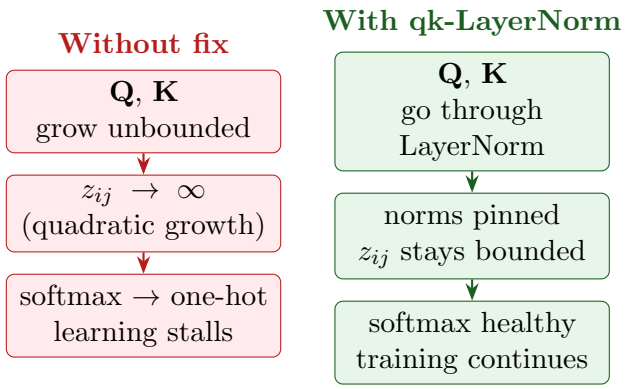
Both q_i and k_j grow linearly with their weight matrix norms, so z_{ij} grows **quadratically** with parameter norm — **much faster** than any other quantity in the network.

What happens next

Once logits are very large, the softmax collapses to a **one-hot distribution**: every token attends to a single fixed position regardless of content. The attention mechanism becomes *content-independent* and learning stalls.

Why quadratic growth is dangerous: All other features in the network have norms that scale *linearly* with parameter norms. The attention logit is the only quantity with a *quadratic* dependence, making it the first to blow up.

The Fix — qk-LayerNorm



Mitigation

qk-LayerNorm: apply LayerNorm to q and k *before* computing the dot product. This pins their norms, **breaking the quadratic dependence** on parameter norm. The instability is completely prevented.

Claim 1 — Curves

Claim 1 (Wortsman et al.)

Attention logit growth appears in small Transformers (<10 M params) at high learning rates, and **qk-layernorm** resolves it, measurably reducing LR sensitivity.

Figure: Claim 1 — Placeholder for Curves

← Insert your Claim 1 figure here: train loss curves + max attention logit over training steps, with/without qk

Suggested content: training loss and max attention logit over training steps, with and without qk-layernorm.

Instability #2 — Output Logit Divergence

First reported in PaLM 540 B (Chowdhery et al., 2022) — reproduced in 2.4 M parameter models in the original paper

Mechanism

Let $y \in \mathbb{R}^V$ be the output logits, giving token probabilities:

$$p_i = \frac{e^{y_i}}{Z}, \qquad Z = \sum_j e^{y_j}$$

Without weight decay, the logits collectively drift toward **very negative values** late in training. As $y_i \to -\infty$:

$Z \to 0 \implies \log p_i = y_i - \log Z$ decouples from y_i

What happens

The cross-entropy loss $-\log p_y$ becomes independent of the logit values: gradients with respect to y vanish, the model receives no useful signal, and training **stalls** even though the loss curve may look deceptively smooth.

This is distinct from attention logit growth: the divergence occurs *towards the end* of training, is *global* (all logits shift), and only manifests when **weight decay is absent**.

The Fix — Z-loss Regularisation

The proposed mitigation (Chowdhery et al., 2022) is an auxiliary loss term:

$$\mathcal{L}_{\text{z-loss}} = \alpha \log^2 Z, \qquad \alpha = 10^{-4}$$

Intuition: When $Z \approx 1$ (i.e. $\log Z \approx 0$), the penalty is near zero. Any drift of Z away from 1 is *softly penalised*, keeping the logits anchored near zero and preventing collective drift to $-\infty$.

Effect on LR sensitivity

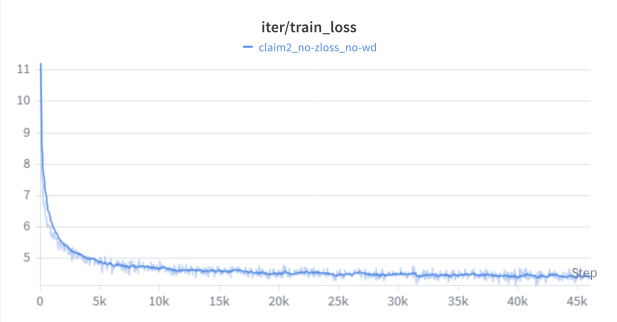
z-loss prevents output logit divergence regardless of model scale, and *weight decay* provides a complementary effect for larger models. Together they keep $\log Z \approx 0$ throughout training.

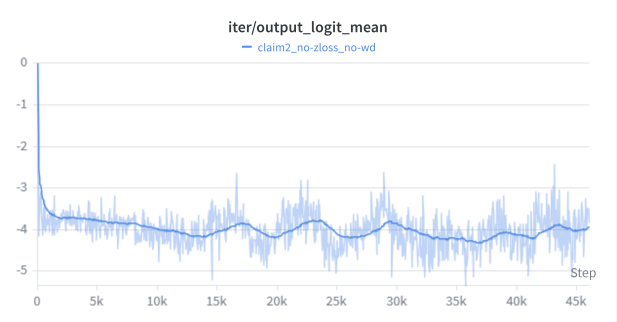
Claim 2 — Curves

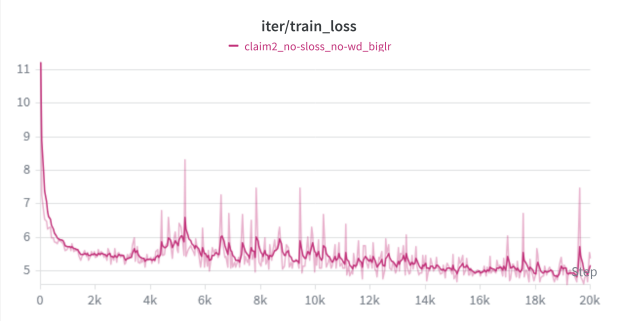
Claim 2 (Wortsman et al.)

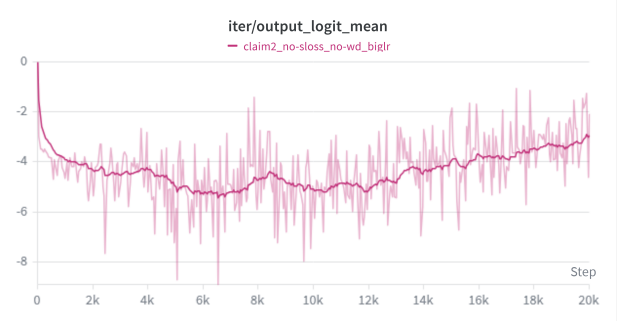
Output logit divergence occurs in Transformer models when weight decay is absent, and **z-loss** prevents this instability. The instability is reproducible in 2.4 M parameter models at high learning rates.

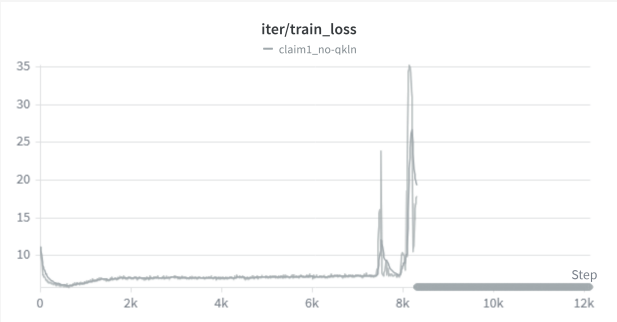
Figure: Claim 2 — Curves from Our Runs

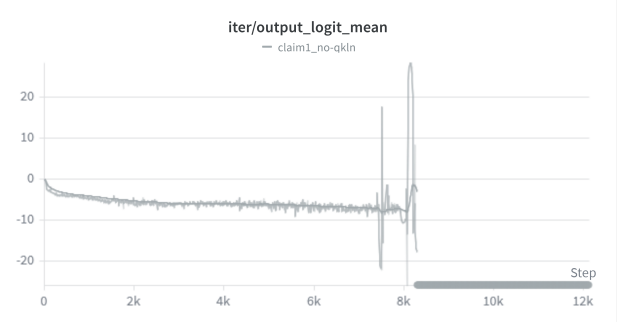












Unstable — train loss

Unstable — logit mean

Claim 2 experiments across three regimes. Top: stable training run ($lr = 0.1$). Middle: moderately stable regime with mild fluctuations (linear warmup to $lr = 0.5$). Bottom: unstable regime showing divergence. In all three cases, we did not observe a collapse of the output logit mean to $-\infty$.

LR Sensitivity — A Unified Metric for Stability

The LR Sensitivity Metric

A model is *fragile* if picking a slightly wrong learning rate leads to significantly worse performance. The paper captures this with **LR Sensitivity**:

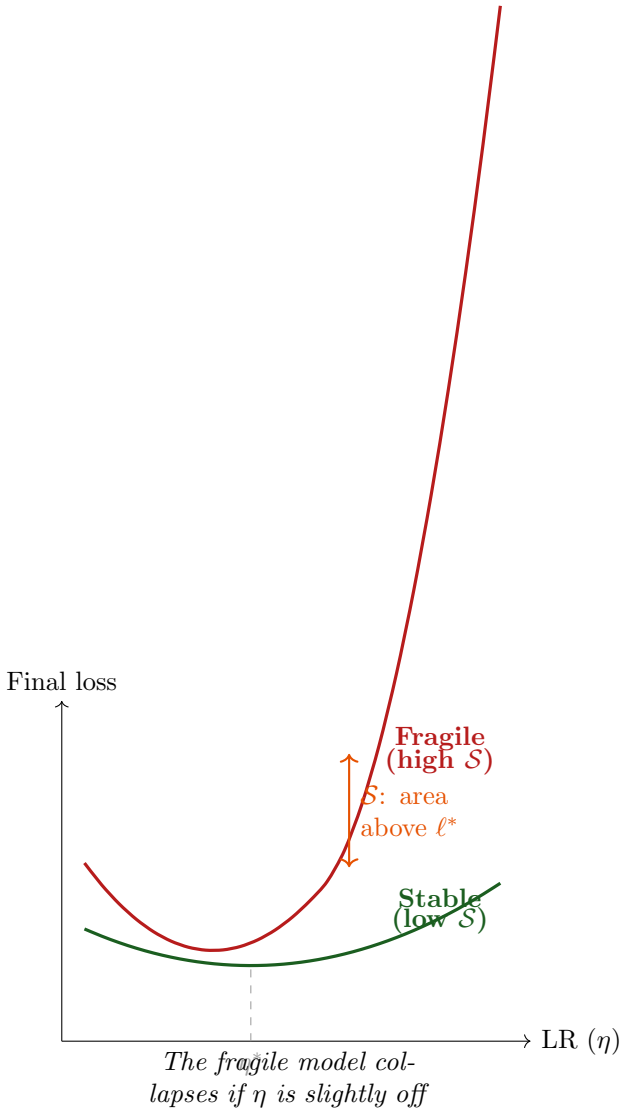
$$\mathcal{S} = \mathbb{E}_{\eta \in [a,b]}\Big[\min\big(\ell(A(\eta)),\, \ell_0\big) - \ell^*\Big]$$

- $A(\eta)$ = model trained with learning rate η
- $\ell^* = \min_\eta \ell(A(\eta))$ = best achievable loss
- ℓ_0 = loss at initialisation (caps diverged runs)
- Range $[a, b]$ spans **3 orders of magnitude**

A high $\mathcal{S}$ means the model requires very careful LR tuning and is likely to diverge if η is even slightly too large.

Key finding

Both qk-layernorm and z-loss **reduce** $\mathcal{S}$. Models with these fixes train stably across 3 orders of magnitude of LR variation. LR sensitivity still *increases with model scale*, but the mitigations slow this growth.



Beyond the Two Instabilities — Paper Insights We Highlight

Warm-up reduces $\mathcal{S}$

Longer LR warm-up substantially reduces LR sensitivity, especially at large scale. Models without warm-up cannot train stably at LR 3×10^{-1}, even with qk-layernorm.

Why it matters: warm-up gives the adaptive optimizer time to build up reliable moment estimates before committing to large gradient steps.

Depth > Width for instability

Scaling *depth* increases LR sensitivity **faster** than scaling width at the same parameter count.

However, the deepest models achieve the *lowest* final loss, suggesting a fundamental trade-off: depth buys performance but costs stability.

Predicting instability early

By tracking the *max attention logit* as a function of model scale (fitted with a quadratic), the paper **predicts** that a 4.8 B model will diverge at LR 10^{-2} before running the experiment — and the prediction is confirmed.

This is a powerful tool: identify failures *cheaply* before they occur at scale.